

```
In [2]: import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

Different types of analysis:

There are different types of analysis as mentioned below.

1.Univariate : In univariate analysis we will be using a single feature to analyze almost of its properties.

2.Bivariate : When we compare the data between exactly 2 features then its called bivariate analysis.

3.Multivariate: Comparing more than 2 variables is called as Multivariate analysis.

list of plots

i) Scatter plot (B)

ii) Pair plot (M)

iii) Box plot (U)

iv) Violin plot(U)

v) Distribution plot (U)

vi) Joint plot(U) & (B)

vii) Bar chart (B)

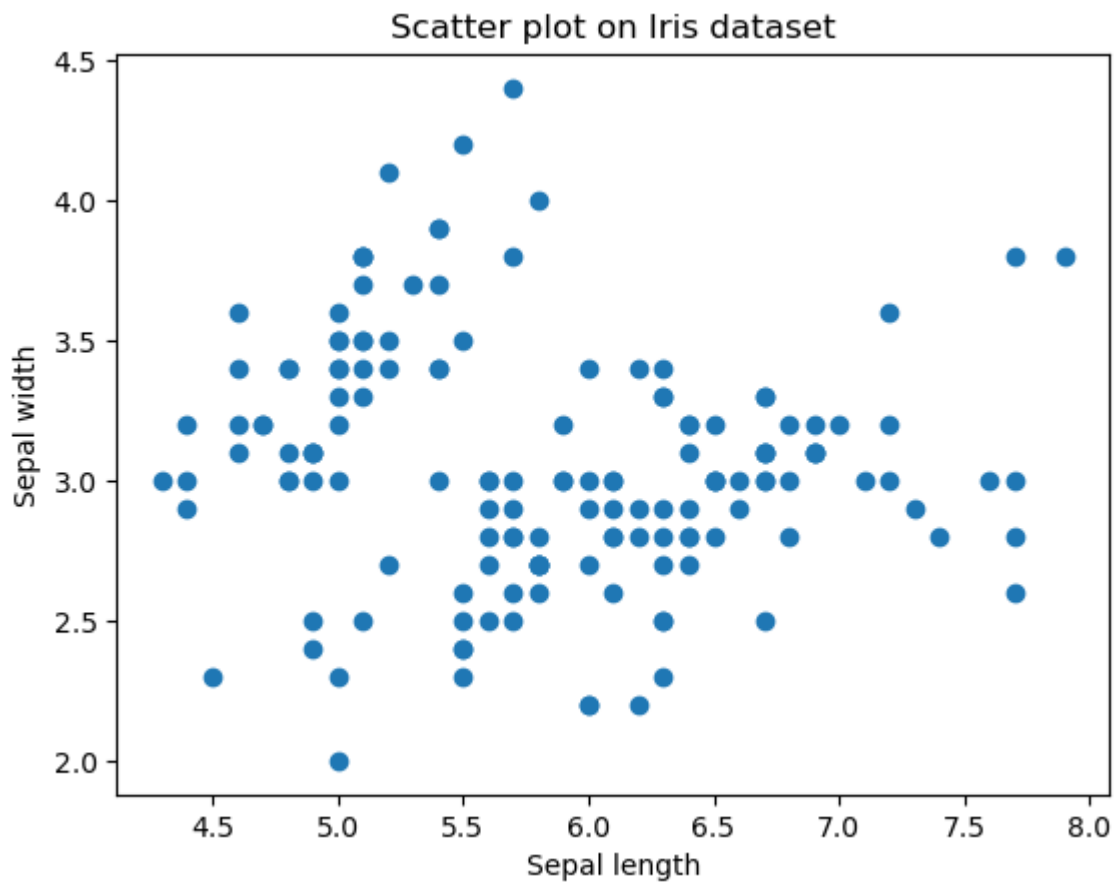
viii) Line plot(B)

Plots for Machine learning for data visualization is scatter plots. Provides a representation of where each points in the entire dataset are present with respect to any 2/3 features(Columns). Useful to find find patterns/Clusters and separability of the data

```
In [6]: #Scatter plot
dfiris=pd.read_csv('/kaggle/input/datasetiris/dataset_iris.csv')
```

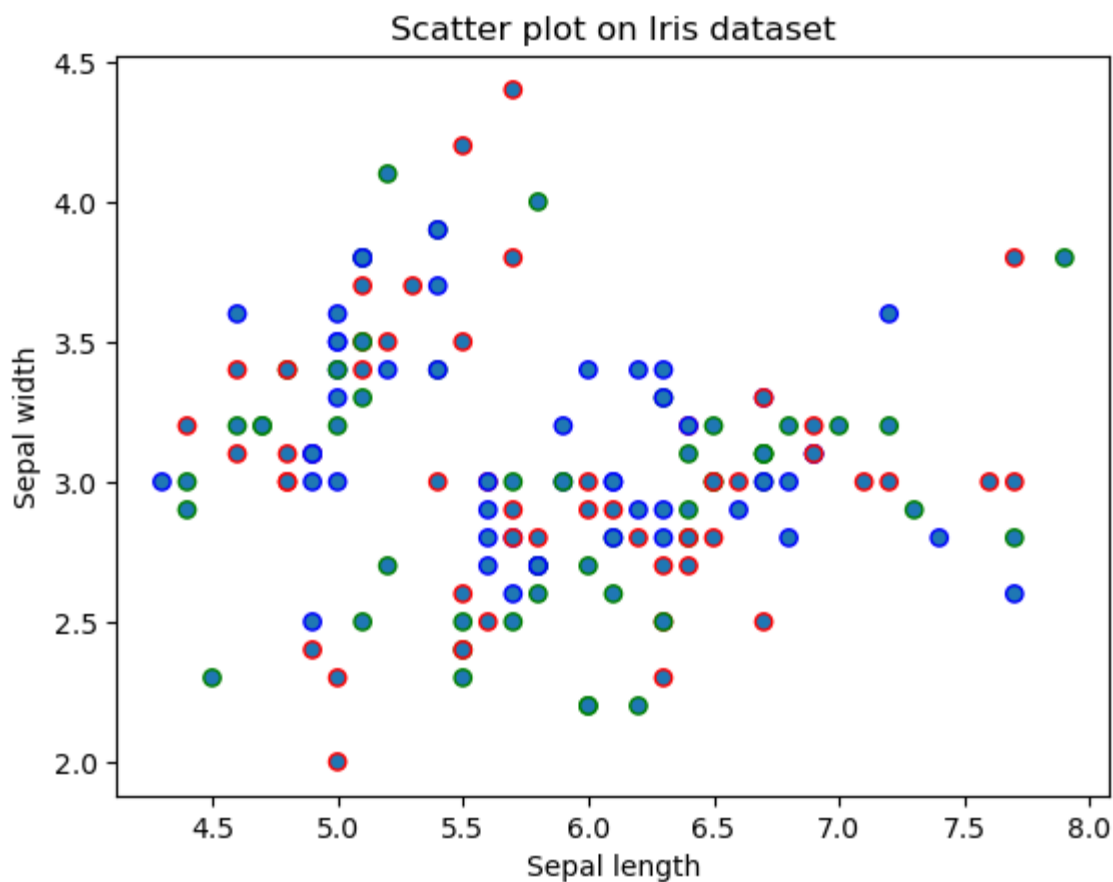
```
In [7]: plt.scatter(dfiris['sepal_length'],dfiris['sepal_width'])
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.title('Scatter plot on Iris dataset')
```

```
Out[7]: Text(0.5, 1.0, 'Scatter plot on Iris dataset')
```



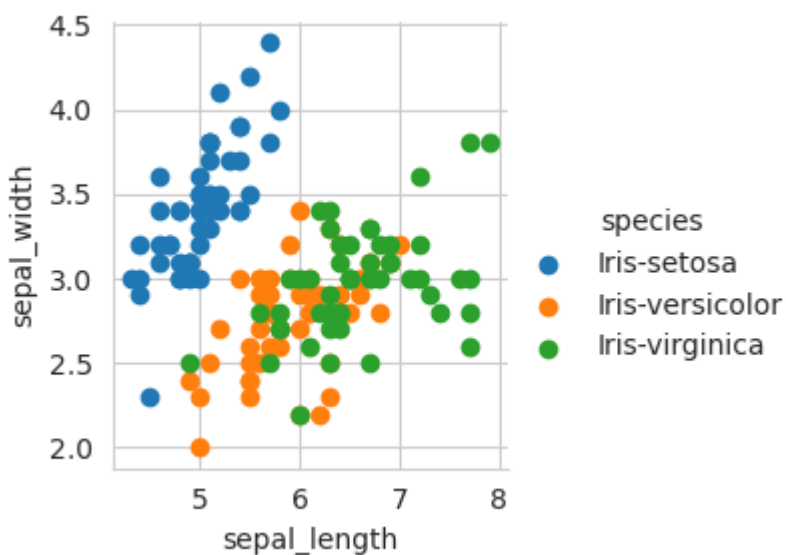
```
In [12]: #different color
plt.scatter(dfiris['sepal_length'],dfiris['sepal_width'],edgecolor=['r','b','g'])
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.title('Scatter plot on Iris dataset')
```

```
Out[12]: Text(0.5, 1.0, 'Scatter plot on Iris dataset')
```

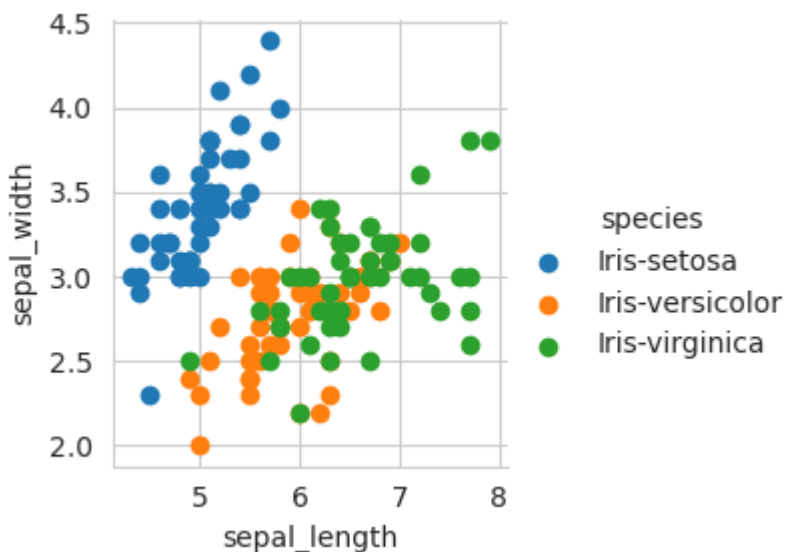


```
In [ ]: # for various species#Here the class labels are Setosa, Virginica and Veriscolor. T
```

```
In [17]: sns.set_style("whitegrid")
sns.FacetGrid(dfiris, hue="species") \
.map(plt.scatter, "sepal_length", "sepal_width") \
.add_legend()
plt.show()
```



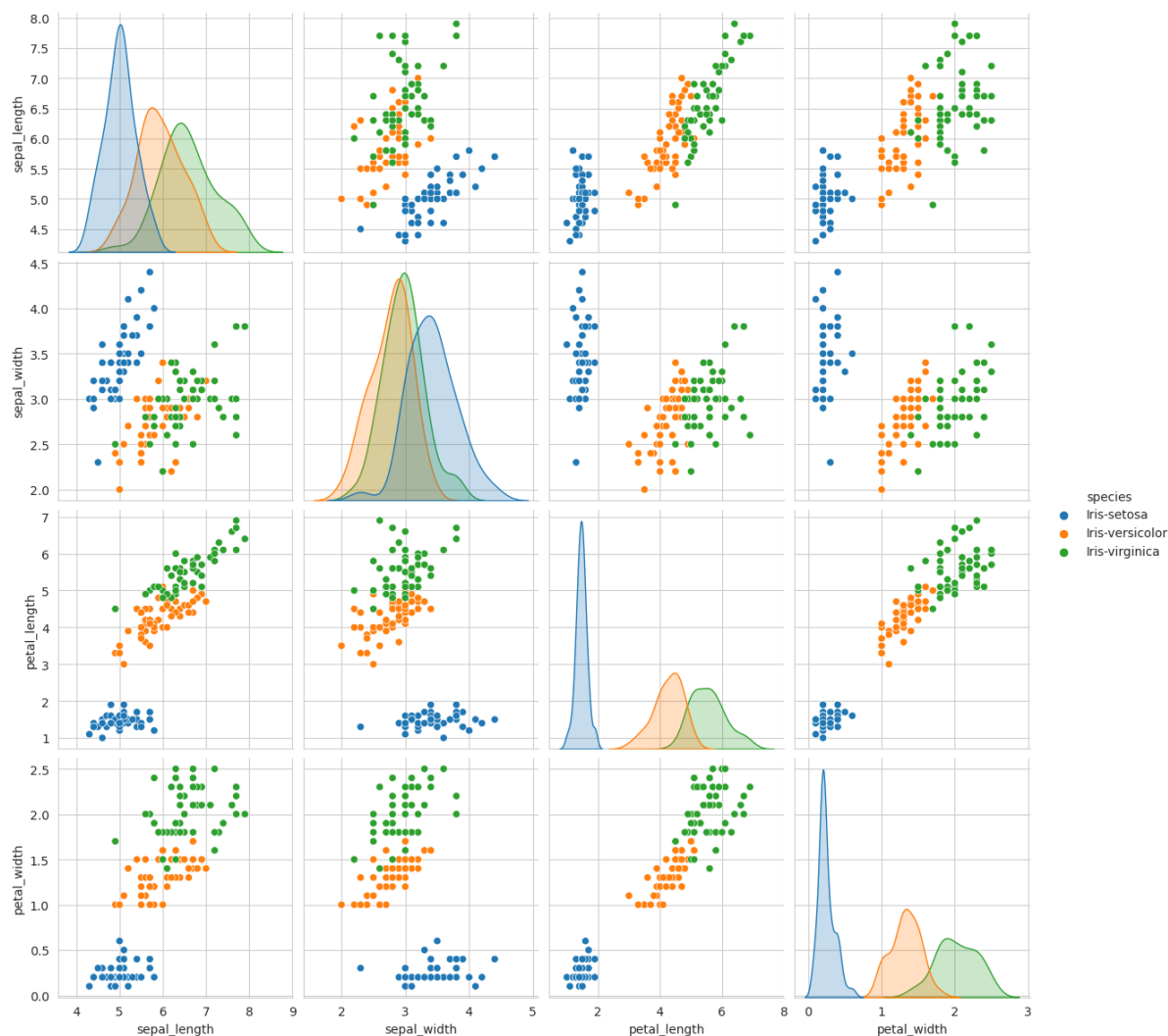
```
In [22]: sns.set_style("whitegrid")
sns.FacetGrid(dfiris, hue="species") \
    .map(plt.scatter, "sepal_length", "sepal_width") \
    .add_legend()
plt.show()
```



```
In [ ]: #The parameter hue, in the facetGrid decides the color of each datapoints.
#Here we used Species column(dependent feature/Class) in hue
```

We can use scatter plots for 2d with Matplotlib and even for 3D, we can use it from plot.ly. When we have 4d or more than that? This is when Pair plot from seaborn package comes into play. Let's say we have n number of features in a data, Pair plot will create us a $(n \times n)$ figure where the diagonal plots will be histogram plot of the feature corresponding to that row and rest of the plots are the combination of feature from each row in y axis and feature from each column in x axis.

```
In [24]: sns.set_style("whitegrid");
sns.pairplot(dfiris, hue="species", height=3);
plt.show()
```



By getting a high level overview of plots from pair plot, we can see which two features can well explain/separate the data and then we can use scatter plot between those 2 features to explore further. From the above plot we can conclude like, Petal length and petal width are the 2 features which can separate the data very well.

pairplot may become complex when we have more number of feature say like 10 or so on. So in such cases, the best will be using a dimensionality reduction technique to map data into 2d plane and visualizing it using a 2d scatter plot.

In [25]: `#Univariate Analysis with Boxplots`

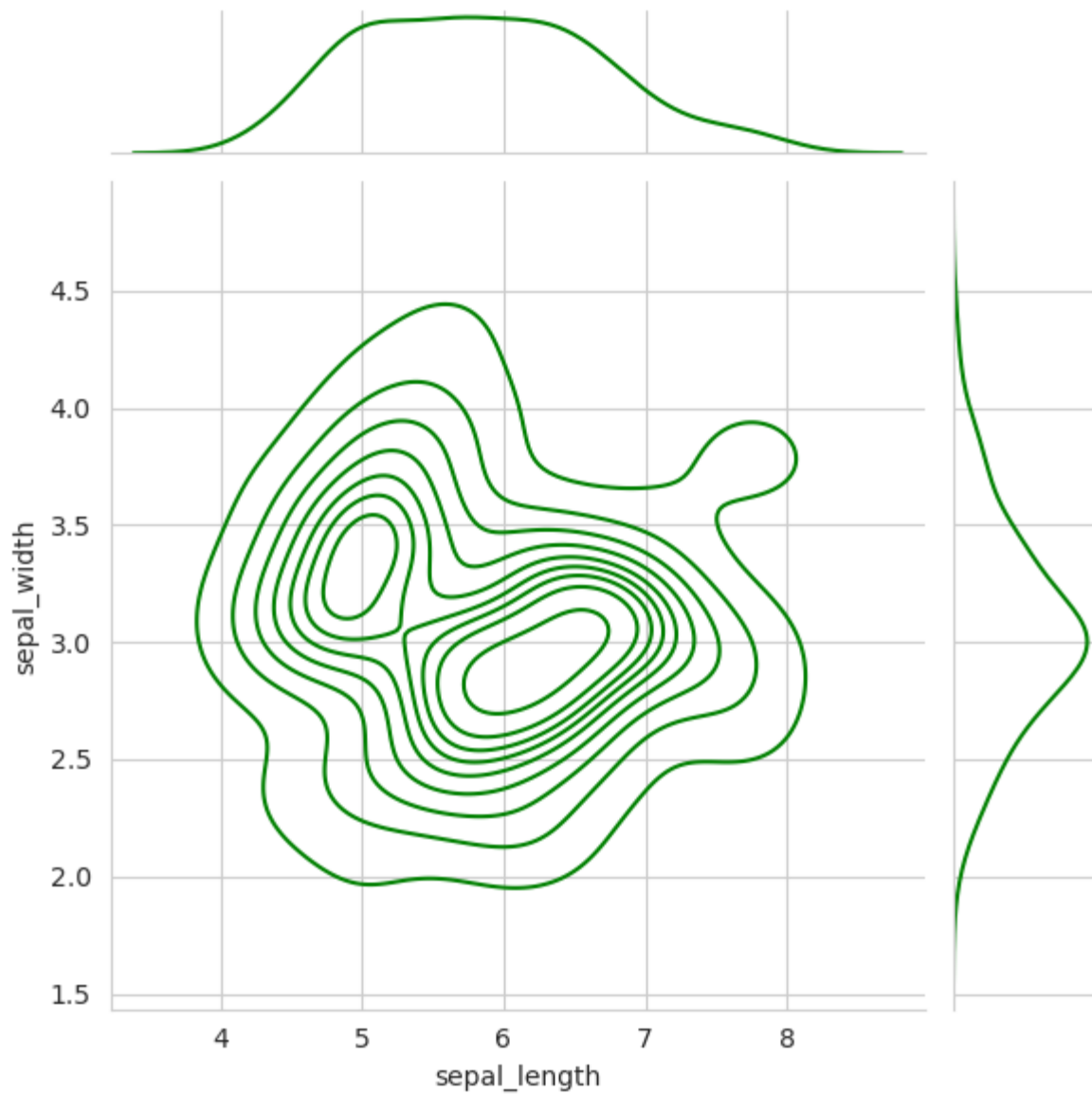
This is the type of plot that can be used to obtain more of the statistical details about the data. The straight lines at the maximum and minimum are also called as whiskers. Points outside of whiskers will be inferred as an outliers. The box plot gives us a representation of 25th, 50th, 75th quartiles. From box plot we can also see the Interquartile range(IQR) where maximum details of the data will be present. It also gives us a clear overview of outlier points in the data.

```
In [ ]: sns.boxplot(x="Species", y="PetalLengthCm", data=iris )  
plt.show()
```

Joint plot : a single figure we can do both univariate as well as bivariate analysis. The main plot will give us a bivariate analysis, whereas on the top and right side we will get univariate plots of both the variables that were considered. The one shown below is of kind as KDE (Kernel Density Estimation) and it is represented in a contour diagram, All points with same boundary will have the same value and the colour at a point depends on number of datapoints i.e it will be light color when very few points had that value and will be darker with more points. This is why at center it is darker and will be pale/lighter at the ends for this dataset.

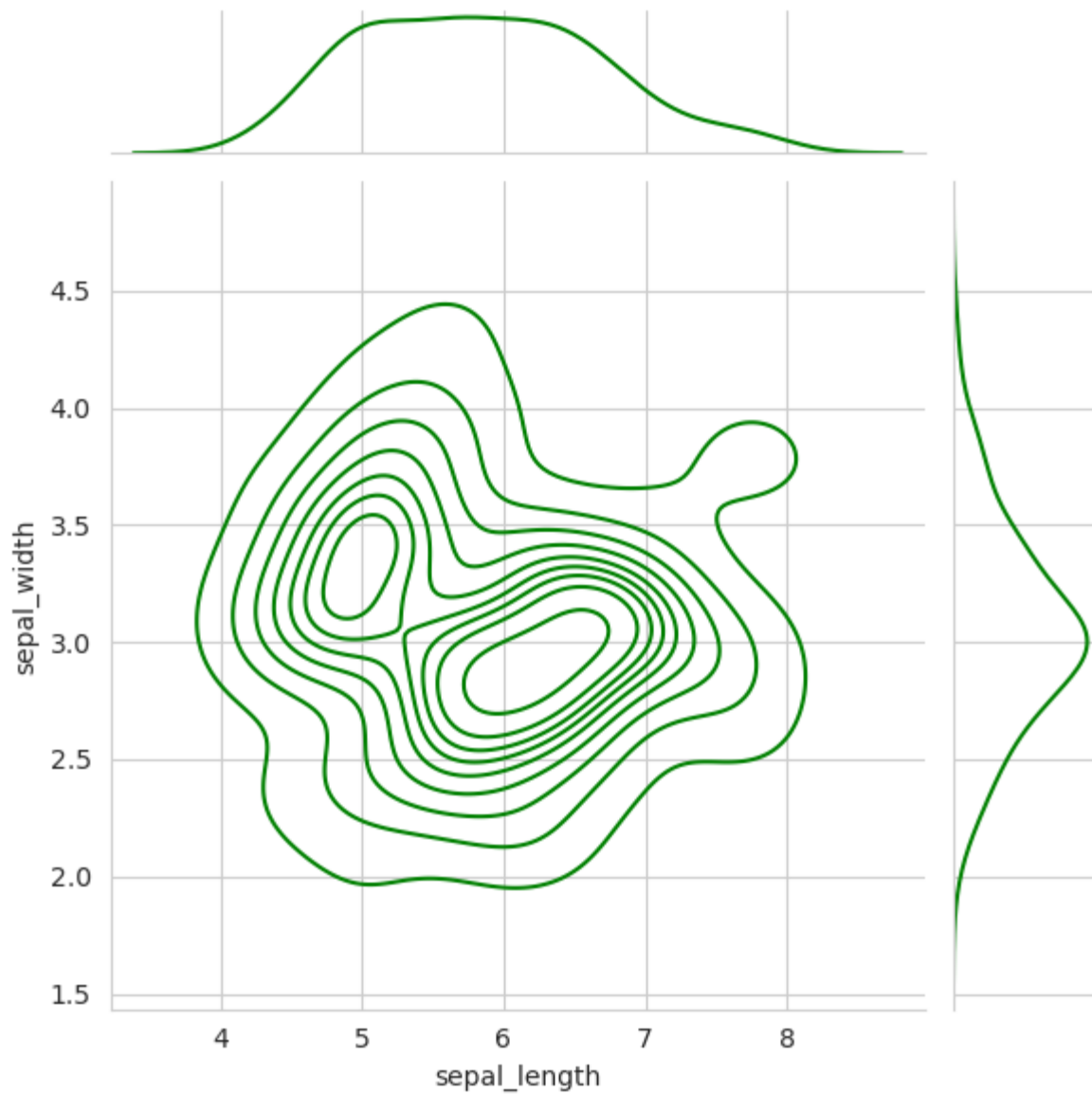
We are getting both Univariate and Bivariate in a single plot as shown below, it will make our work a lot easier.

```
In [35]: sns.jointplot(data=dfiris, x=dfiris['sepal_length'], y=dfiris['sepal_width'], kind='kd')  
Out[35]: <seaborn.axisgrid.JointGrid at 0x7f6da26b7110>
```



```
In [40]: sns.jointplot(data=dfiris,x=dfiris['sepal_length'],y=dfiris['sepal_width'],kind='kde')
```

```
Out[40]: <seaborn.axisgrid.JointGrid at 0x7f6da2eec190>
```

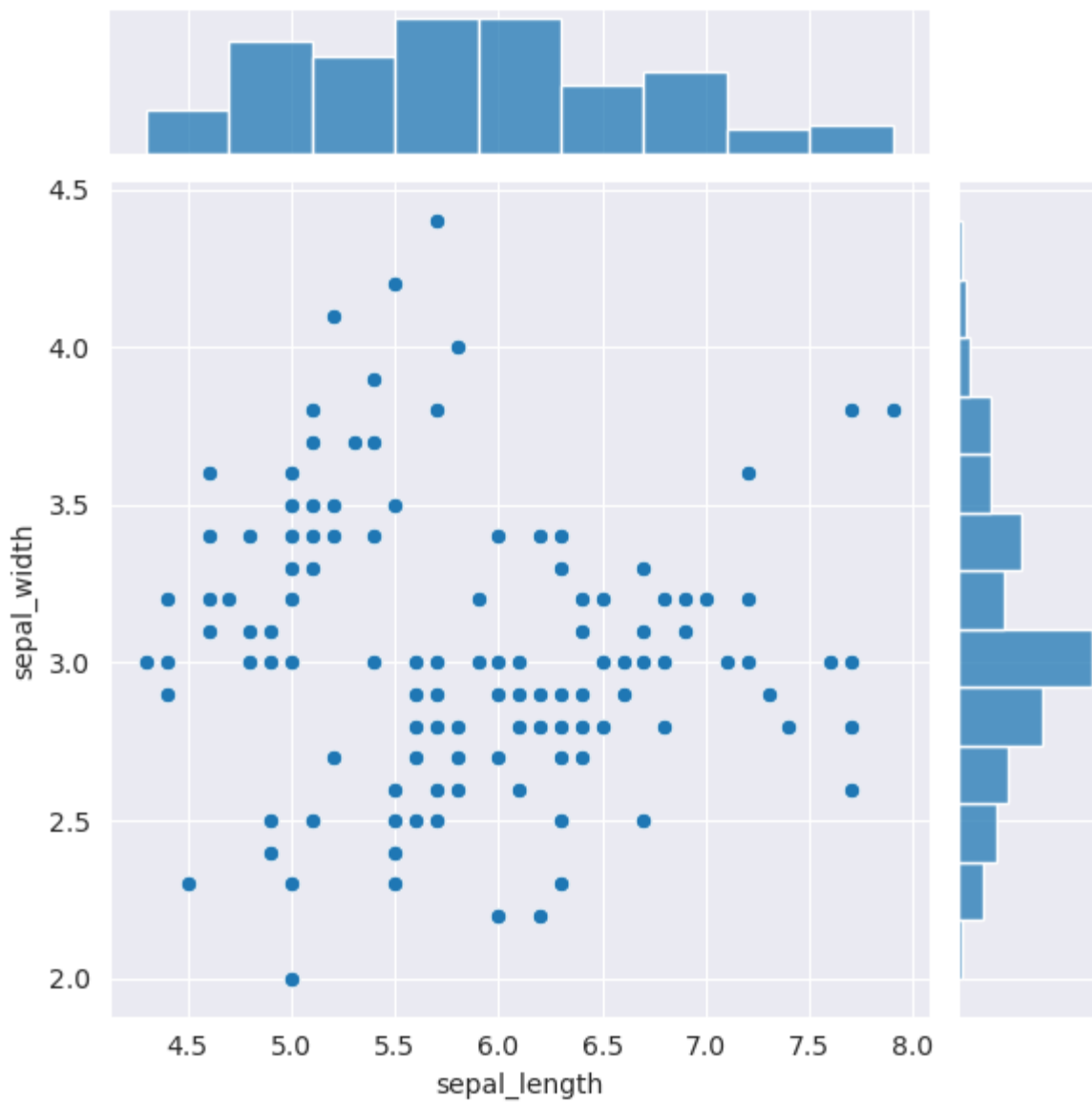


In the middle is the relationship figure and is a bivariate relationship Histogram gives a view of what x distribution looks like Joint distribution of data Imagine the y pushing the data over right and looking at the

```
In [41]: sns.set_style('darkgrid')
```

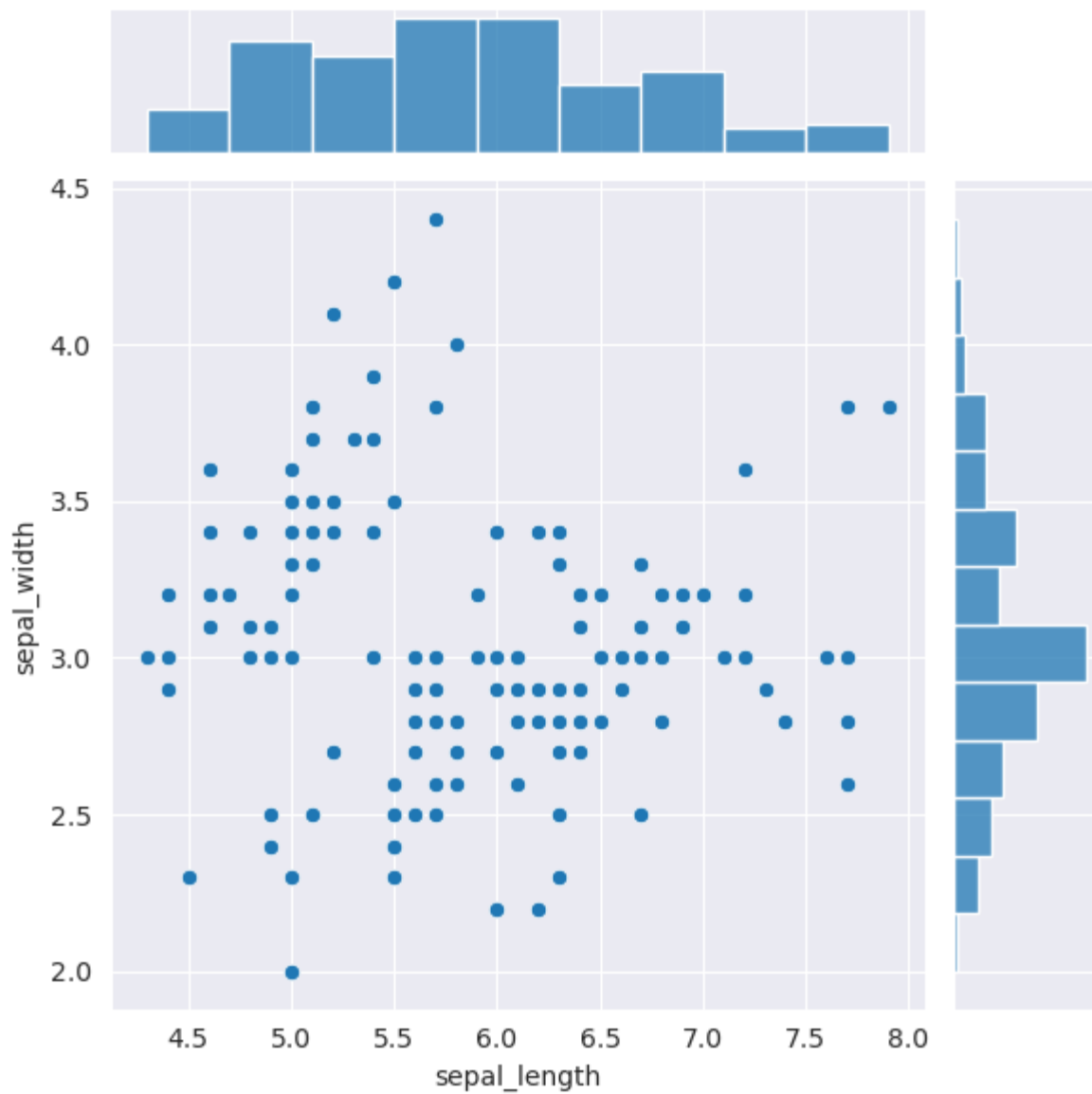
```
In [43]: sns.jointplot(x=dfiris['sepal_length'],y=dfiris['sepal_width'])
```

```
Out[43]: <seaborn.axisgrid.JointGrid at 0x7f6da2acb650>
```

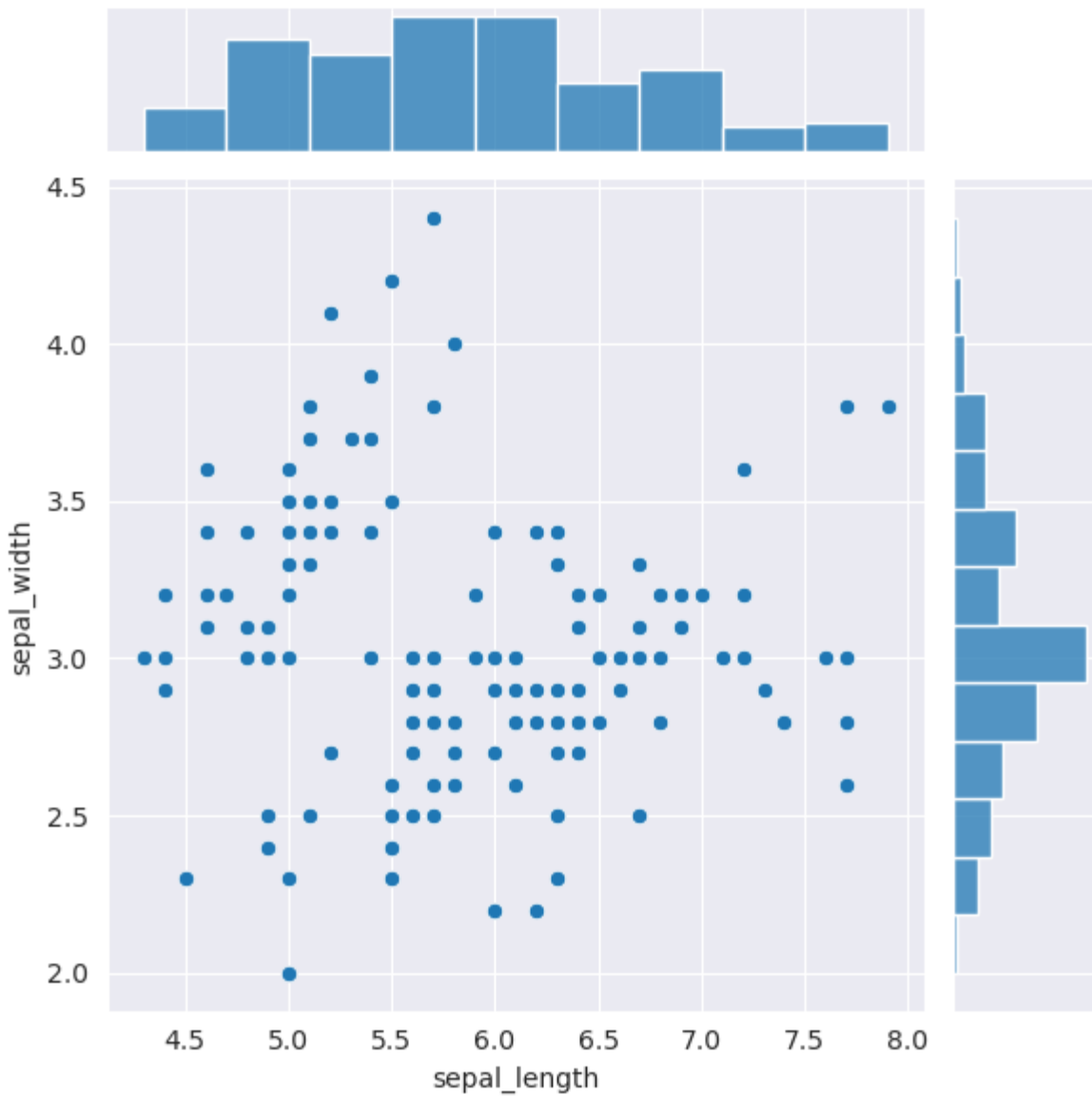
```
In [44]: #sns.jointplot(x=dfiris['sepal_length'],y=dfiris['sepal_width'])  
sns.jointplot(x='sepal_length',y='sepal_width',data=dfiris)
```

```
Out[44]: <seaborn.axisgrid.JointGrid at 0x7f6da2a46490>
```



```
In [ ]: sns.jointplot(x=dfiris['sepal_length'],y=dfiris['sepal_width'],kind='kde')
```

```
In [45]: g=sns.jointplot(x=dfiris['sepal_length'],y=dfiris['sepal_width'])
```



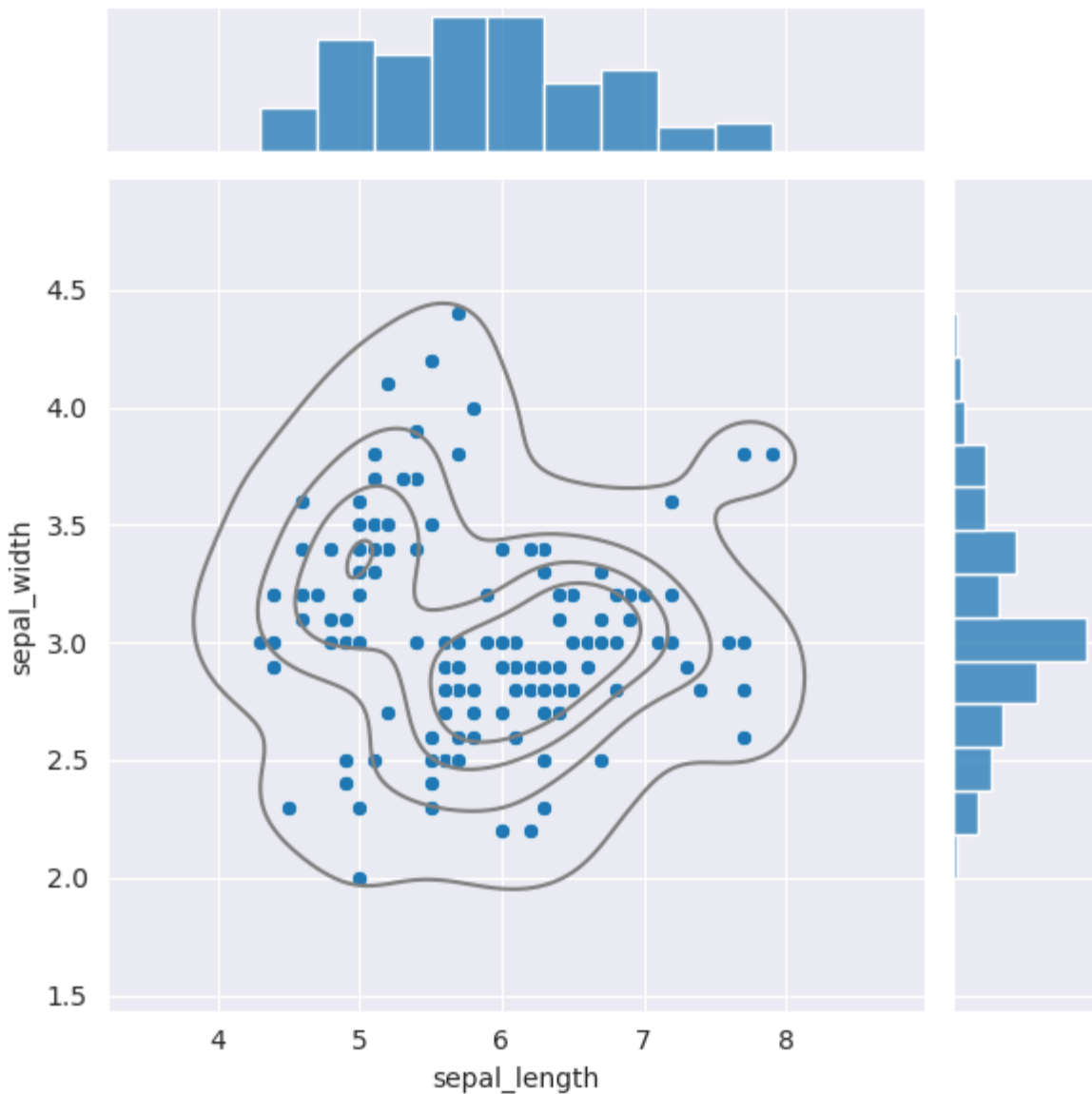
```
In [46]: type(g)
```

```
Out[46]: seaborn.axisgrid.JointGrid
```

You can take these and continue adding more

```
In [48]: g=sns.jointplot(x=dfiris['sepal_length'],y=dfiris['sepal_width'])
         g.plot_joint(sns.kdeplot,color='gray',levels=5)
```

```
Out[48]: <seaborn.axisgrid.JointGrid at 0x7f6da18f0050>
```



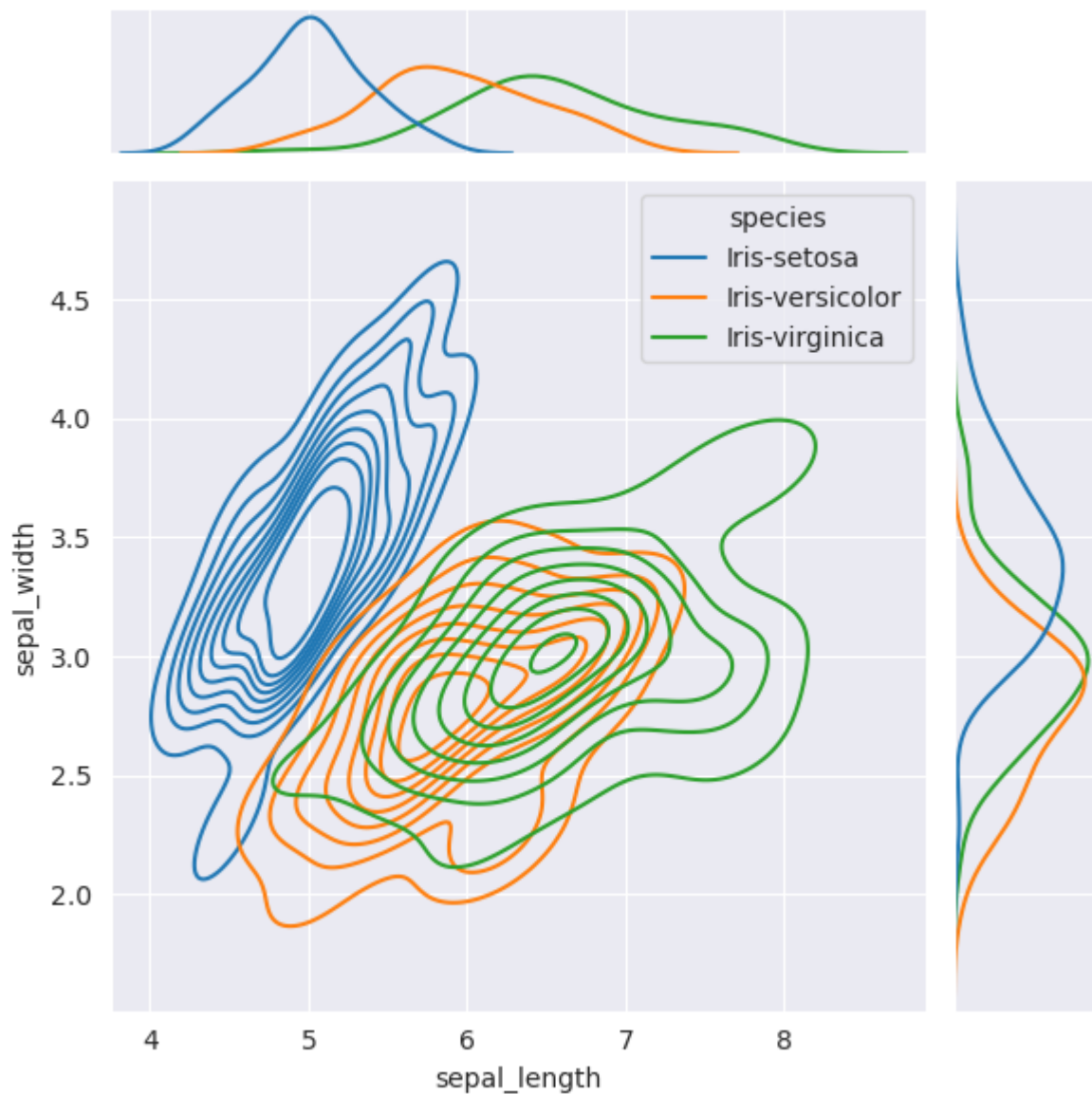
What happens if we have a categorical variable

In [55]: `dfiris.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   sepal_length 150 non-null   float64
1   sepal_width  150 non-null   float64
2   petal_length 150 non-null   float64
3   petal_width  150 non-null   float64
4   species      150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

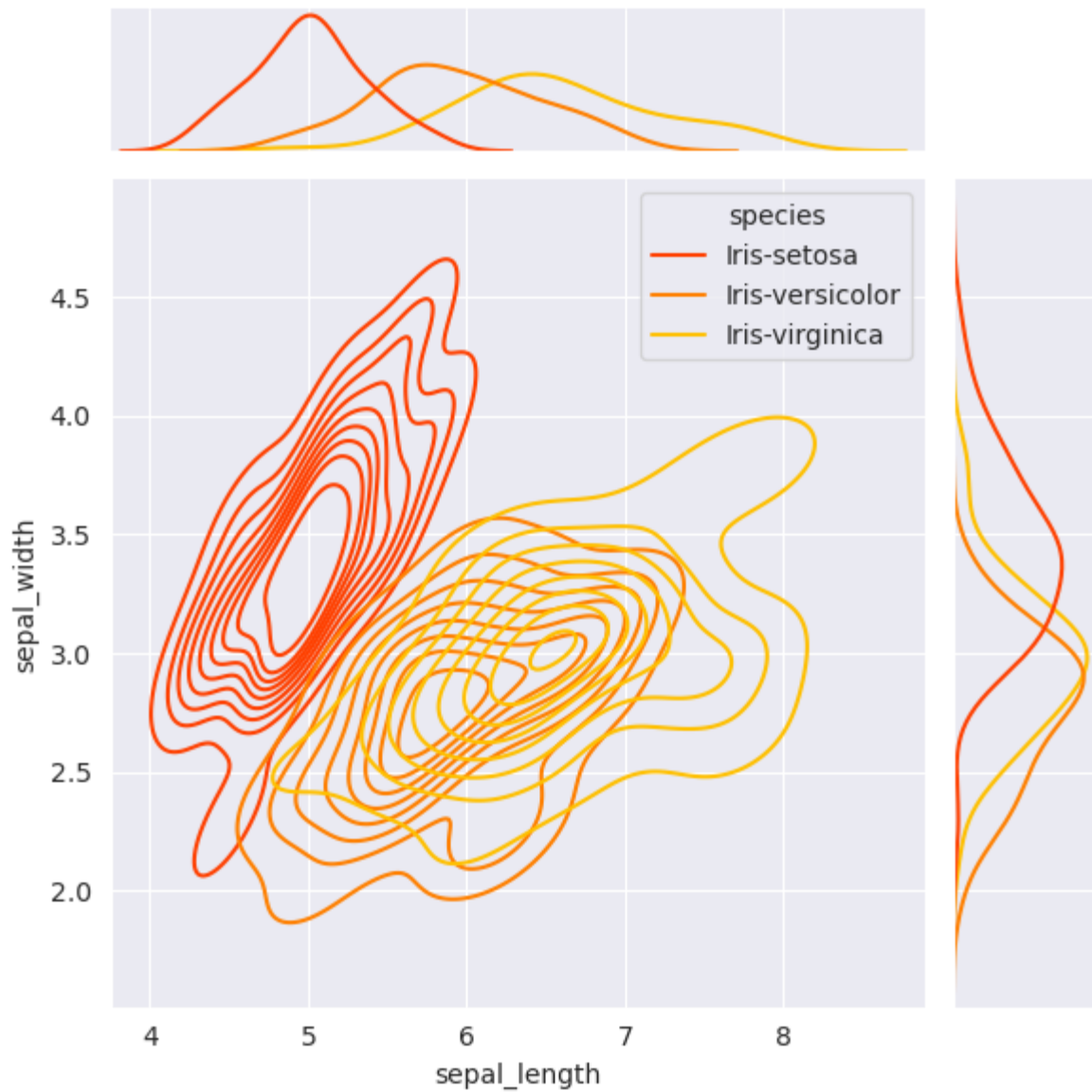
In [56]: `#hue for kind column`
`sns.jointplot(x=dfiris['sepal_length'],y=dfiris['sepal_width'],hue=dfiris['species'])`

Out[56]: <seaborn.axisgrid.JointGrid at 0x7f6da0076250>



```
In [57]: #https://www.youtube.com/watch?v=Hey1EAtgduQ  
#hue for kind column and palette for set of color  
sns.jointplot(x=dfiris['sepal_length'],y=dfiris['sepal_width'],hue=dfiris['species']
```

Out[57]: <seaborn.axisgrid.JointGrid at 0x7f6d995a57d0>



```
In [59]: #https://www.youtube.com/watch?v=Hey1EAtgduQ  
#hue for kind column and palette for set of color  
sns.jointplot(x=dfiris['sepal_length'],y=dfiris['sepal_width'],hue=dfiris['species'])
```

```
Out[59]: <seaborn.axisgrid.JointGrid at 0x7f6d99479e50>
```

